

Cheat sheet Java OCP 21

Mỗi chương một trang

Oracle Certified Professional: Java SE 21 Developer (1Z0-830)

Tất cả ví dụ và câu hỏi được kiểm chứng bằng OpenJDK 21.0.10 · Bản build 2026-09-28

Không tài liệu nào bảo đảm điểm tuyệt đối — hãy dùng sách để hiểu Java, không để học thuộc.

Kiểm tra font tiếng Việt: ă ằ ẳ ặ ơ ư ơ đ ơ Đ

Cheat sheet 1 — Dữ liệu: số, boolean, chuỗi, ngày giờ

Primitive: byte(8) short(16) char(16, không dấu) int(32) long(64, L) float(32, f) double(64) boolean. Literal: 1_000, 0x1F, 017, 0b101; _ chỉ giữa hai chữ số. Field có giá trị mặc định, biến cục bộ thì không.

Quy tắc	Ví dụ
byte/short/char + ... → ít nhất int	byte c = a + b; ✗ · b += 1; ✓
Hằng compile-time vừa kiểu thì gán được	final int k = 10; byte b = k; ✓ · float f = 1.0; ✗
Cast thu hẹp cắt bit / cắt phần lẻ	(byte) 200 = -56 · (int) -3.9 = -3
Không “nới rộng rồi boxing”	Long x = 5; ✗ · long y = 5; ✓
Cache Integer -128..127	== đúng với 127, sai với 128 → dùng equals
Long.equals(42)	false (42 là Integer)
Unboxing null	NullPointerException

Biểu thức: trái → phải, rồi mới ưu tiên. i++ + ++i (i = 5) = 12. && / || short-circuit, &/|| thì không. 1 + 2 + "3" + 4 = "334". Chia nguyên cho 0 → ArithmeticException; số thực → Infinity/NaN.

Math: round(-2.5) = -2 (trả long; với float trả int) · floor/ceil/rint trả double · abs(MIN_VALUE) < 0 · floorDiv(-7,2) = -4, floorMod(-7,2) = 1, -7 / 2 = -3, -7 % 2 = -1 · addExact ném khi tràn.

String (bất biến): substring(b, e) không lấy e · indexOf → -1 · strip() (Unicode) vs trim() · isBlank · repeat · split bỏ chuỗi rỗng cuối · pool: "ja" + "va" == "java" ✓, biến không final + literal ✗ · intern(). compareTo: hiệu ký tự đầu khác nhau, hoặc hiệu độ dài.

StringBuilder (mutable): append/insert/delete/deleteCharAt/replace/reverse/setLength trả về chính nó; equals = so sánh tham chiếu; substring trả String.

Text block: """ + xuống dòng · bỏ thụt lề chung (tính cả """" đóng) · bỏ khoảng trắng cuối dòng · \s giữ 1 space · \ cuối dòng = nối dòng · """" đóng riêng dòng → có \n cuối.

java.time (bất biến): quên gán lại d.plusDays(1) là bẫy · LocalDate.of(2023,2,29) → DateTimeException · 31/1 + 1 tháng = 28/29/2 · Period (Y/M/D, P1Y2M3D, không tự normalize ngày) vs Duration (giây, PT25H) · Period.ofYears(1).ofMonths(3) = P3M (static!) · LocalDate.plus(Duration) → UnsupportedTemporalTypeException · DST gap: 02:30 không tồn tại → 03:30; overlap: chọn offset sớm; plusDays(1) giữ giờ địa phương, plusHours(24) giữ thời gian thật · isEqual so sánh instant, equals so sánh cả zone.

Nguồn tham khảo (Sources): tóm tắt từ [chương 1](#) — mọi quy tắc đã được kiểm chứng bằng code chạy trên JDK 21.0.10; link gốc ở mục Sources của chương.

Cheat sheet 2 — Luồng điều khiển

Switch	case X:	case X ->
Statement	fall-through, break	không fall-through
Expression	yield v; mỗi nhánh; phải đầy đủ	biểu thức / { ... yield v; } / throw; phải đầy đủ

- Không trộn : và -> trong một switch. default có thể ở giữa — vẫn fall-through xuống dưới.
- Selector cổ điển: char, byte, short, int (+ wrapper), String, enum. **Không** long/float/double/boolean.
- Nhân cổ điển: hằng compile-time, không trùng. Java 21: case Color.RED (tên đầy đủ) được phép.
- **Phải đầy đủ** khi: switch expression, hoặc switch có pattern / case null. Đầy đủ = default, hoặc đủ hằng enum, hoặc đủ lớp con sealed, hoặc pattern không điều kiện.
- **Pattern switch:** case Integer i when i > 0 ->; thứ tự hẹp → rộng; case có guard trước case cùng kiểu không guard; default không được đứng trước case pattern. null → NPE trừ khi có case null (case null, default ->).
- **Record pattern:** case Point(int x, var y) ->, lồng nhau được; instanceof Line(Point a, Point b).

Flow scoping (instanceof pattern):

Viết	Biến pattern dùng được?
if (o instanceof String s && s.isEmpty())	✓ về phải
if (o instanceof String s s.isEmpty())	✗
if (!(o instanceof String s)) return; s.length();	✓ sau if
str instanceof String t (cùng kiểu)	✓ Java 21 (Java 17: lỗi)

Vòng lặp: for(;;) vô hạn · for (int i = 0, j = 5; ...; i++, j--) · for-each: biến là **bản sao** · do { } while (c); chạy ≥ 1 lần · continue trong for → update, trong while/do → kiểm tra điều kiện · while (n++ < 3) → n = 4 · break LABEL / continue LABEL cho vòng lồng; break trong switch chỉ thoát switch.

Unreachable = lỗi: lệnh sau break/continue/return/throw · sau while(true) không break · thân while(false) · if (false) { } **không** lỗi. if (x = 5) với int x ✗; if (b = true) với boolean b ✓. else thuộc if gần nhất.

Nguồn tham khảo (Sources): tóm tắt từ [chương 2](#) — mọi quy tắc đã được kiểm chứng bằng code chạy trên JDK 21.0.10; link gốc ở mục Sources của chương.

Cheat sheet 3 — Hướng đối tượng

Đa hình: method instance → theo **object** (lúc chạy). Field + method static → theo **kiểu biến** (lúc biên dịch). Gọi method bị override từ constructor cha → field của con còn mặc định (0/null).

Khởi tạo: static (cha → con, một lần, theo thứ tự xuất hiện) → mỗi new: [cha: field/khởi instance → constructor] → [con: ...]. this(...) hoặc super(...) phải là dòng đầu; không cả hai. Instance initializer chỉ chạy trong constructor gọi super. Viết constructor bất kỳ → mất default constructor. Cha không có no-arg → con phải super(args). Field final gán đúng 1 lần trong mọi constructor.

Overload: exact/widening → boxing → varargs; cụ thể nhất; không cái nào cụ thể hơn → ambiguous (f(null) với String/StringBuilder). Chỉ khác kiểu trả về → không phải overload.

Override: cùng chữ ký · quyền không hẹp hơn · trả về covariant · không thêm checked exception rộng hơn · không override final/static (static = hiding)/private. equals(MyType) là **overload**. Override equals phải override hashCode.

Record: field private final + accessor x() + equals/hashCode/toString (dùng field) · không field instance khác · ngầm final, không extends · compact constructor R { x = ...; } gán **tham số**, không this.x · canonical constructor không hẹp quyền hơn record · constructor khác phải this(...) · shallow immutable.

Nested: static nested new Outer.N() · inner outer.new I(), Outer.this.x · local/anonymous: biến cục bộ effectively final. GC: object không còn reachable; System.gc() chỉ là gợi ý.

var: chỉ biến cục bộ có initializer · **X** var x; var x = null; var a = 1, b = 2; var arr = {1}; var f = () -> 1; var a[] · **X** field, tham số, kiểu trả về · var không phải keyword.

Sealed: sealed ... permits A, B · lớp con final | sealed | non-sealed (record/enum ngầm final) · cùng module/package.

Cast: upcast tự động · downcast sai → ClassCastException · lớp không liên quan → lỗi biên dịch · cast sang interface OK trừ khi lớp final không implements · null instanceof X = false.

Abstract: không new · method abstract không thân, không final/private/static · lớp cụ thể phải cài đặt hết.

Interface: field public static final · method abstract public · default · static (gọi I.m()), không kế thừa · private · hai default trùng → phải override, A.super.m() · lớp thẳng interface · default + abstract cùng chữ ký từ 2 interface → lỗi · functional interface: đúng 1 abstract (không tính method Object).

Enum: hằng số đứng đầu, ; nếu có thêm thành viên · constructor ngầm private (public **X**) · values(), valueOf (phân biệt hoa thường, sai → IllegalArgumentException), ordinal(), name(), compareTo · method abstract → mọi hằng phải cài đặt · implements được, extends **X**, new **X**.

Nguồn tham khảo (Sources): tóm tắt từ [chương 3](#) — mọi quy tắc đã được kiểm chứng bằng code chạy trên JDK 21.0.10; link gốc ở mục Sources của chương.

Cheat sheet 4 — Ngoại lệ

Throwable — Error (unchecked)
└─ Exception (checked) — RuntimeException (unchecked): NPE, ArithmeticException, ClassCastException, IllegalArgumentException ← NumberFormatException, IllegalStateException, UnsupportedOperationException, IndexOutOfBoundsException ← ArrayIndexOutOfBoundsException

- **Checked:** handle (catch) or declare (throws). Lớp kế thừa trực tiếp Throwable cũng là checked.
 - try thường cần ≥ 1 catch hoặc finally; {} bắt buộc; try-with-resources không cần catch/finally.
 - Catch: lớp con **trước** lớp cha (ngược lại → “already been caught”). Catch checked exception mà try không thể ném → lỗi (trừ Exception/Throwable). Unchecked: catch thoải mái.
 - **Multi-catch** A | B e: A, B không cha-con; e ngằm final.
 - **try-with-resources:** AutoCloseable; đóng **ngược** thứ tự, **trước** catch/finally; biến ngằm final, phạm vi trong try; Java 9 try (r) với biến effectively final; tài nguyên khởi tạo lỗi → chỉ đóng các tài nguyên đã tạo; exception của close() → getSuppressed() của exception chính.
 - **finally:** luôn chạy (trừ System.exit). return x chốt giá trị trước finally (primitive không đổi; object vẫn bị sửa). return/throw trong finally **thay thế** kết quả/exception (mất luôn, không suppressed).
 - **Custom:** extends Exception (checked) / RuntimeException (unchecked); constructor (String msg, Throwable cause). getMessage(); toString() = tên.lớp.đầy.đủ: msg (lớp lồng dùng \$); getCause().
 - **Override:** được bỏ throws, ném hẹp hơn, thêm unchecked; không thêm checked mới/rộng hơn.
 - **Precise rethrow:** catch (Exception e) { throw e; } với e effectively final → compiler chỉ tính exception thật của try.
 - Static init lỗi → ExceptionInInitializerError (lần sau: NoClassDefFoundError).
 - Không bắt → stack trace ra System.err, thread kết thúc.
 - Lambda: chỉ ném checked exception mà method của functional interface khai báo (Callable được, Runnable/Supplier không).
- Nguồn tham khảo (Sources):** tóm tắt từ [chương 4](#) — mọi quy tắc đã được kiểm chứng bằng code chạy trên JDK 21.0.10; link gốc ở mục Sources của chương.

Cheat sheet 5 — Mảng và Collections

Mảng: `new int[3]` (có size) · `new int[]{1,2}` (không size) · `{1,2}` chỉ khi khai báo · `int[] a, b[]` → `b` là 2 chiều · `a.length` · `new int[2][]` jagged · `Arrays.sort` (String: số < HOA < thường) · `binarySearch` cần sắp xếp, không thấy → `-(chèn)-1` · `equals/compare/mismatch` · `asList`: fixed-size, set ghi xuống mảng, add ✗ · `Arrays.asList(int[]) = List<int[]>`.

	Thứ tự	null
<code>ArrayList/LinkedList</code>	chỉ số	✓
<code>HashSet / LinkedHashSet / TreeSet</code>	không / chèn / sắp xếp	1 / 1 / ✗
<code>HashMap / LinkedHashMap / TreeMap</code>	không / chèn / sắp xếp key	key null 1 / 1 / ✗
<code>ArrayDeque</code>	hai đầu	✗
<code>List.of, Set.of, Map.of, copyOf</code>	bất biến	✗ (Set/Map trùng → IAE)

List: `add(i, e)` chèn · `set` thay, trả cũ · `remove(int)` vs `remove(Object)` · `subList = view` · `removeIf`, `replaceAll` · sửa khi `forEach` → thường `ConcurrentModificationException` · `Collections.unmodifiableList = view`, `List.copyOf = bản sao`.

Set: add trả false nếu có · `TreeSet`: `floor ≤`, `ceiling ≥`, `lower <`, `higher >`, `headSet(x) <x`, `tailSet(x) ≥x`, `pollFirst` · `comparator` trả 0 = trùng.

Map: `put` trả cũ · `putIfAbsent` trả hiện tại · `getOrDefault` không thêm · `merge(k,v,f)` (`f` → null: xóa) · `compute*` · `keySet/values/entrySet` views · `TreeMap`: `firstKey` (rỗng → exception), `floorKey`, `headMap`, `tailMap(k, incl)` · key mutable bị sửa → không tìm thấy.

Deque: `push/pop/peek` ở đầu (stack) · `offer/add cuối`, `poll/remove/peek đầu` (queue) · exception (`add/remove/element/getFirst`) vs giá trị đặc biệt (`offer/poll/peek`).

Sequenced (Java 21): `addFirst/addLast/getFirst/getLast/removeFirst/removeLast/reversed()` (view) · `Map`: `putFirst`, `firstEntry`, `sequencedKeySet` · bất biến → UOE.

Sắp xếp: `Comparable.compareTo` (tự nhiên) · `Comparator.comparing(f).thenComparing(g).reversed()` (`reversed` đảo phần trước nó) · `comparingInt` · `naturalOrder/reverseOrder/nullsFirst/nullsLast` · `list.sort(null) = tự nhiên` · `Collections.max/min/frequency/nCopies/reverse`.

Generics: `invariant(List<Number> = new ArrayList<Integer>()) ✗` · `? extends T` đọc, không add (trừ null) · `? super T` add T, đọc ra Object (PECS) · không primitive · diamond chỉ sau `new` · type erasure.

Nguồn tham khảo (Sources): tóm tắt từ [chương 5](#) — mọi quy tắc đã được kiểm chứng bằng code chạy trên JDK 21.0.10; link gốc ở mục Sources của chương.

Cheat sheet 6 — Lambda và Stream

Lambda: `x -> ...` · `(a, b) -> ...` · `(String s) -> ...` / `(var s) -> ...` (không trộn) · khối `{ return ...; }` · biến cục bộ: `final/effectively final` (gán lại ở **bất kỳ** đâu → lỗi) · tham số không trùng tên biến cục bộ · `() -> { return; }` chỉ khớp void · method reference không có `()`.

Method ref	Ví dụ
static	<code>Integer.parseInt</code>
object cụ thể	<code>System.out.println</code>
instance, object = tham số đầu	<code>String.length</code> , <code>String.compareTo</code>
constructor	<code>ArrayList.new</code> , <code>int[].new</code>

Interface	Method		Interface	Method
<code>Supplier<T></code>	<code>get()</code>		<code>Function<T,R></code>	<code>apply (+ andThen, compose)</code>
<code>Consumer<T></code>	<code>accept (+ andThen)</code>		<code>UnaryOperator<T></code>	<code>apply</code>
<code>Predicate<T></code>	<code>test (+ and/or/negate/not)</code>		<code>BinaryOperator<T></code>	<code>apply</code>
<code>BiFunction<T,U,R></code>	<code>apply</code>		<code>IntSupplier</code>	<code>getAsInt</code>

Stream: nguồn → intermediate (lười) → **một** terminal · không terminal → không chạy (kể cả `peek`) · dùng lại → `IllegalStateException` · từng phần tử đi hết pipeline (trừ `sorted`, `stateful`) · short-circuit: `limit`, `findFirst`, `anyMatch`, `takeWhile`.

Tạo: `Stream.of`, `Arrays.stream`, `iterate(seed, f)` (vô hạn) / `iterate(seed, hasNext, f)`, `generate`, `concat`, `IntStream.range` (không gồm cuối) / `rangeClosed`, `Stream.of(int[]) = 1` phần tử!

Terminal: `toList()` (bất biến) · `collect` · `reduce(id, op) → T` · `reduce(op) → Optional` · `reduce(id, acc, comb)` · `count()` → `long` · `min/max(cmp)` → `Optional` · `findFirst/findAny` · `any/all/noneMatch` (rỗng: `false/true/true`) · `forEach(Ordered)`.

Primitive: `sum()` (`int/long/double`) · `average()` → `OptionalDouble` · `max()` → `OptionalInt` · `summaryStatistics` · `boxed`, `mapToObj`, `mapToInt` · `IntStream` không có `toList()`.

Optional: `of` (`null` → NPE), `ofNullable`, `empty` · `get/orElseThrow()` rỗng → `NoSuchElementException` · `orElse(x)` **luôn** tính x, `orElseGet(s)` lười · `map/flatMap/filter/or/ifPresentOrElse/stream`.

Collectors: `toList/toSet/toCollection` · `joining(d, p, s)` · `toMap(k, v)` trùng → ISE; `toMap(k, v, merge, TreeMap::new)` · `groupingBy(f)` (`HashMap`, `key null` → NPE) · `groupingBy(f, mapFactory, downstream)` · `partitioningBy(p)` luôn có `false & true` · `counting` (`Long`), `summingInt`, `averagingInt` (`Double`), `maxBy` (`Optional`), `mapping`, `filtering`, `flatMaping`, `teeing`.

Parallel: `parallelStream()` / `.parallel()`; cuối cùng `parallel/sequential` quyết định · `reduce` đúng khi identity trung hoà + hàm kết hợp · nguồn có thứ tự: `collect/toList/findFirst/forEachOrdered` giữ thứ tự; `forEach/findAny` thì không.

Nguồn tham khảo (Sources): tóm tắt từ [chương 6](#) — mọi quy tắc đã được kiểm chứng bằng code chạy trên JDK 21.0.10; link gốc ở mục Sources của chương.

Cheat sheet 7 — Module, JAR, jlink

```
module com.shop {
    requires java.sql;           // module-info.java ở GỐC thư mục module, không có package
    requires transitive com.base; // đọc (reads) module khác
    requires static com.opt;     // ai requires com.shop cũng đọc com.base
    exports com.shop.api;       // bắt buộc lúc biên dịch, tùy chọn lúc chạy
    exports com.shop.spi to com.plugin; // package cho mọi module
    opens com.shop.model;       // qualified export
    opens com.shop.dto to com.fw; // deep reflection lúc chạy (cả private)
    uses com.shop.api.Payment;  // qualified opens
    provides com.shop.api.Payment with com.shop.impl.Card; // consumer
}                               // provider (Interface with Impl1, Impl2)
```

- `java.base` requires ngầm (mandated) · không vòng requires · không split package (1 package ↔ 1 module) · `exports` package rỗng/không tồn tại → lỗi · `open module` → mọi package mở, bên trong không được `opens`.
- `public` không đủ: phải `exports` · reflection private cần `opens` → không có: `InaccessibleObjectException`.
- **Service:** interface (exports) + provider (requires api, provides ... with ..., impl cần public no-arg constructor hoặc public static provider(), không cần export) + consumer (requires api, uses, `ServiceLoader.load(S.class): findFirst(), stream(), iterator()`). Thiếu uses → `ServiceConfigurationError` **lúc chạy**. Không provider → rỗng.

	Named	Automatic	Unnamed
Ở đâu	module path + module-info	module path, JAR không module-info	classpath
Tên	trong module-info	Automatic-Module-Name hoặc từ tên file (my-utils-2.0.1.jar → my.utils)	không có
Đọc	requires (+transitive)	mọi module + unnamed	mọi module
Export	exports	tất cả package	tất cả, nhưng named module không requires được

JAR có module-info đặt trên classpath → bị coi là unnamed. **Migration:** bottom-up (thư viện lá trước) / top-down (ứng dụng trước, thư viện làm automatic module).

Việc	Lệnh
Biên dịch nhiều module	<code>javac -d out --module-source-path src -m a,b</code> (thêm <code>-p mods</code> nếu phụ thuộc module có sẵn)
Chạy	<code>java -p out -m mod/pkg.Main --module-path, --module</code>
JAR modular	<code>jar --create --file m.jar --main-class p.Main -C out/mod . (-c -f -e); chạy java -p dir -m mod</code>
Mô tả	<code>java -p out --describe-module mod · jar --describe-module --file m.jar (-d) · java --list-modules</code>
Phụ thuộc	<code>jdeps -summary x.jar (-s) · --list-deps · --print-module-deps · --jdk-internals</code>
Runtime image	<code>jlink --module-path out --add-modules mod --output img</code> (gồm phụ thuộc bắc cầu)

Nguồn tham khảo (Sources): tóm tắt từ [chương 7](#) — mọi quy tắc đã được kiểm chứng bằng code chạy trên JDK 21.0.10; link gốc ở mục Sources của chương.

Cheat sheet 8 — Đồng thời

Tạo thread: `extends Thread + start()` · `new Thread(runnable[, name])` · `Thread.ofPlatform().name("p").start(r)` · `Thread.ofVirtual().start(r)` / `.unstarted(r)` · `Thread.startVirtualThread(r)` · `Executors.newVirtualThreadPerTaskExecutor()`. Virtual thread: luôn daemon (`setDaemon(false)` → IAE), rẻ, hợp việc chờ I/O. `run()` trực tiếp = không có thread mới; `start()` 2 lần → `IllegalThreadStateException`.

Trạng thái: NEW → RUNNABLE → BLOCKED (chờ monitor) / WAITING (`join()`, `wait()`) / TIMED_WAITING (`sleep`, `join(ms)`) → TERMINATED. `interrupt()`: đánh thức sleep/wait/join bằng `InterruptedException`, hoặc chỉ đặt cờ.

Runnable	Callable<V>
<code>void run()</code> , không checked exception	<code>V call()</code> throws Exception
<code>execute(r)</code> (void), <code>submit(r)</code> → <code>Future<?></code> (<code>get = null</code>)	<code>submit(c)</code> → <code>Future<V></code> , <code>invokeAll</code> , <code>invokeAny</code>

ExecutorService: `newSingleThreadExecutor`, `newFixedThreadPool(n)`, `newCachedThreadPool`, `newScheduledThreadPool` (`schedule`, `scheduleAtFixedRate`, `scheduleWithFixedDelay`) · `invokeAll` giữ thứ tự task · `invokeAny` trả **kết quả** của một task thành công (tất cả lỗi → `ExecutionException`) · `shutdown()` (không nhận mới → `RejectedExecutionException`) · `shutdownNow()` (interrupt, trả task chưa chạy) · `awaitTermination` · `isShutdown/isTerminated` · `close()` (Java 19+, try-with-resources, đợi xong). **Future:** `get()` chặn; lỗi task → `ExecutionException` (`getCause()`) · `get(t, u)` → `TimeoutException` · `cancel`, `isDone`, `state()`.

Thread-safety: `count++` không nguyên tử · `synchronized` method (this / Class nếu static — **hai khoá khác nhau**) · `synchronized` (obj) obj phải là tham chiếu; constructor không `synchronized` · khoá nội tại `reentrant` · `wait/notify` ngoài `synchronized` → `IllegalMonitorStateException`. `ReentrantLock`: `lock()` (chờ) + `unlock()` trong **finally** · `tryLock()` / `tryLock(t, u)` → boolean · `hold count` · `unlock` khi không giữ → IMSE · `ReentrantReadWriteLock`. `Atomic`: `getAndIncrement` (cũ) vs `incrementAndGet` (mới), `compareAndSet`, `updateAndGet`, `accumulateAndGet`, `LongAdder`. `CountDownLatch` (không âm, một lần) · `CyclicBarrier` (đùng lại). `Deadlock` / `starvation` / `livelock`.

Collections: `ConcurrentHashMap` (không null; `merge/compute` nguyên tử) · `CopyOnWriteArrayList` (iterator snapshot, không CME) · `BlockingQueue`: `put/take` chặn, `offer/poll` không · `Collections.synchronizedList` (duyệt vẫn tự khoá).

Parallel stream: `common ForkJoinPool` · không side effect; dùng `collect`, `groupingByConcurrent` · `forEachOrdered` giữ thứ tự · `reduce`: identity trung hoà + kết hợp.

Nguồn tham khảo (Sources): tóm tắt từ [chương 8](#) — mọi quy tắc đã được kiểm chứng bằng code chạy trên JDK 21.0.10; link gốc ở mục Sources của chương.

Cheat sheet 9 — I/O và NIO.2

Byte	Char	Ghi chú
FileInputStream / FileOutputStream(f, append)	FileReader / FileWriter(f, cs, append)	mặc định ghi đè
BufferedInputStream/OutputStream	BufferedReader (readLine → null) / BufferedWriter (newline)	
DataInputStream/DataOutputStream	—	readInt ... hết → EOFException; int 4, long 8 byte
ObjectInputStream/ObjectOutputStream	—	readObject() ném IOException + ClassNotFoundException
—	PrintWriter / PrintStream	không ném IOException (checkError)
InputStreamReader / OutputStreamWriter		cầu nối byte ↔ char; chọn charset

read() → int, hết = -1 · read(buf) → số byte đọc · skip, mark/reset (markSupported), readAllBytes, transferTo · UTF-8: "é" 2 byte, "ê" 3 byte · System.console() có thể **null**; readPassword() → char[].

Serialization: implements Serializable · field không serializable → NotSerializableException · transient, static không lưu · deserialize: constructor/initializer của lớp Serializable **không** chạy; no-arg constructor của lớp cha đầu tiên **không** Serializable **có** chạy · serialVersionUID khác → InvalidClassException · record dùng canonical constructor · object dùng chung/vòng tham chiếu được giữ nguyên trong một stream.

Path (chỉ thao tác chuỗi): Path.of("a", "b"), Paths.get, file.toPath() · new Path **X** · getFileName, getParent, getRoot (null nếu tương đối), getNameCount/getName(i) (không tính gốc), subpath(b, e) · resolve(abs) → abs · relativize trộn tuyệt đối/tương đối → IAE · normalize() bỏ . và x/..; /.. → / · equals/startsWith/endsWith theo thành phần (endsWith(".java") false) · toRealPath cần file tồn tại.

Files (đa số ném IOException): - exists/notExists/isDirectory/isRegularFile/isSymbolicLink/isSameFile/isHidden/size - createFile (có rồi → FileAlreadyExistsException) · createDirectory (cha thiếu → NoSuchFileException) · createDirectories (không lỗi) - readString, readAllLines (List), lines (Stream, **đồng!**), writeString, write(p, lines) (mỗi dòng + \n), newBufferedReader/Writer - copy (đích có → FAEE trừ REPLACE_EXISTING), move (nguồn mất), delete (không có → NSFE; thư mục không rỗng → DirectoryNotEmptyException), deleteIfExists → boolean - list (1 cấp, không gồm start) · walk(start[, depth]) (gồm start) · find(start, depth, (p, attr) -> ...) - readAttributes(p, BasicFileAttributes.class), getLastModifiedTime, LinkOption.NOFOLLOW_LINKS - StandardOpenOption: mặc định ghi = CREATE, TRUNCATE_EXISTING, WRITE; APPEND không CREATE trên file chưa có → NSFE; CREATE_NEW.

Nguồn tham khảo (Sources): tóm tắt từ [chương 9](#) — mọi quy tắc đã được kiểm chứng bằng code chạy trên JDK 21.0.10; link gốc ở mục Sources của chương.

Cheat sheet 10 — Bản địa hoá

Locale: `Locale.of("vi","VN")` (Java 19+; `new Locale` deprecated) · `Locale.US`, `Locale.FRANCE`, `Locale.GERMAN` (chỉ ngôn ngữ) ≠ `Locale.GERMANY` · `forLanguageTag("vi-VN")` · `new Locale.Builder()` · `toString()` → `vi_VN`, `toLanguageTag()` → `vi-VN` · `getCountry()` có thể "" · `getDisplayName(inLocale)` = tên bằng ngôn ngữ của `inLocale` · `setDefault([Category,] 1): FORMAT` (số/ngày) vs `DISPLAY` (tên).

ResourceBundle: `getBundle("Msg", fr_CA)` với mặc định `en_US` thử: `Msg_fr_CA` → `Msg_fr` → `Msg_en_US` → `Msg_en` → `Msg`. Chọn bundle **đầu tiên** tìm được (lớp Java trước `.properties`). Key thiếu → tìm lên **cha của bundle đã chọn** (`fr_CA` → `fr` → gốc), không tìm ở locale mặc định. Không thấy → `MissingResourceException` (bundle hoặc key). Key phân biệt hoa thường. `.properties` đọc UTF-8. `getString` trên giá trị không phải `String` → `ClassCastException`; `getObject`, `keySet`, `getLocale`. `ListResourceBundle.getContents()` → `Object[][]`.

NumberFormat (abstract; dùng factory): `getInstance/getNumberInstance` (≤ 3 chữ số lẻ) · `getIntegerInstance` · `getCurrencyInstance` (\$1,234.57, số âm -\$5.00) · `getPercentInstance` (×100) · `getCompactNumberInstance`(1, `SHORT|LONG`) (2M, 2 million) · làm tròn **HALF_EVEN** (12.5 → 12, 13.5 → 14, 0.125 → 12%) · `setMin/MaxFractionDigits` · `parse` → `Number` (`Long/Double`), dùng ở ký tự lạ ("12abc" → 12), không đọc được → `ParseException` (checked) · US "1,5" → 15; DE "1.500,75" → 1500.75. **DecimalFormat:** 0 bắt buộc, # tùy chọn, , nhóm, . thập phân, %, 'text', pos;neg (#,##0.00;(#,##0.00)).

DateTimeFormatter: `ofPattern(p[, locale])` · `ofLocalizedDate(FormatStyle.SHORT|MEDIUM|LONG|FULL).withLocale(l)` (US: 12/25/24, Dec 25, 2024, December 25, 2024) · ISO hằng số · `date.format(f) = f.format(date)` · `LocalDate.parse(s, f)` sai → `DateTimeParseException`; `parse` phân biệt hoa thường · pattern có giờ cho `LocalDate` → `UnsupportedTemporalTypeException`.

y năm	M/MM/MMM/MMMM tháng	d/dd ngày	D ngày trong năm	E/EEEE thứ
H/HH 0-23	h/hh 1-12 + a	m/mm phút	s giây	'...' chữ, '' nháy

MessageFormat: `format("{0} có {1}", a, b)` · `{0,number}`, `{0,number,#.##}`, `{0,number,percent}`, `{0,number,integer}`, `{0,date,short}` · '{0}' in nguyên văn, '' → ' ' · thiếu tham số → `in {n}` · `new MessageFormat(p, locale)` · `String.format(Locale, "%,.2f", x)`.

Nguồn tham khảo (Sources): tóm tắt từ [chương 10](#) — mọi quy tắc đã được kiểm chứng bằng code chạy trên JDK 21.0.10; link gốc ở mục Sources của chương.